2. Life-before-devops

Wednesday, July 15, 2026 4:06 PM

1. How Software Was Built Traditionally

Before DevOps, software teams followed a **strictly separated model**:

- Developers wrote code
- Operations deployed and maintained it
- Communication happened late
- Both teams worked in isolation

This approach worked when:

- releases were rare
- software changed slowly
- systems were small

But as software grew, this model started breaking.

2. The Traditional Dev vs Ops Split

Developers (Dev):

- write features
- push code
- move to the next task

Operations (Ops):

- deploy code
- manage servers
- keep systems stable

The Core Conflict:

- Dev wants **speed**
- Ops wants **stability**

Their goals naturally clashed, creating friction.

3. The "Works on My Machine" Problem

A classic issue in traditional development:

"It works on my machine."

Why it happened:

- Dev environment ≠ Production environment
- Manual setup steps
- No standardized environments

Result:

- broken deployments
- wasted time
- frustration across teams

4. Manual Deployments Everywhere

Deployments used to be:

- manual
- slow
- error-prone
- stressful

Typical old deployment flow:

- copy files manually

- run commands by hand
- restart servers manually
- hope nothing breaks

This made deployments **rare, risky, and scary**.

5. Slow and Infrequent Releases

Because deployments were risky:

- releases happened once every **weeks or months**
- bug fixes took long to reach users
- feedback cycles were slow

This slowed down:

- innovation
- learning
- business growth

6. Blame Culture Instead of Ownership

When production broke:

- Dev: “It worked in my environment.”
- Ops: “Your code is bad.”

Impact:

- teams blamed each other
- trust decreased
- morale dropped
- no one owned the system end-to-end

7. Scaling Made Everything Worse

As applications grew:

- more users
- more servers
- more complexity

Traditional workflows couldn’t scale:

- manual steps failed
- human errors increased
- downtime became expensive

Companies needed a **new model**.

8. Business Pressure Increased

Modern businesses demanded:

- faster feature delivery
- high availability
- minimal downtime
- quick rollback

The old model couldn’t keep up with these expectations.

9. Summary of Core Problems

Traditional software delivery suffered from:

- siloed teams
- slow releases
- manual deployments
- inconsistent environments
- lack of ownership
- poor communication
- high failure rates

These weren’t rare issues — they were **normal**.

10. Why These Problems Couldn’t Be Ignored

Ignoring these problems led to:

- unhappy users
- lost revenue
- burned-out engineers
- unstable systems

This pain forced companies to rethink how software should be built and operated.

And That Rethink Led to DevOps

DevOps emerged as the solution to:

- break silos
- automate workflows
- standardize environments
- increase release frequency
- improve reliability
- create shared ownership